Universität Passau

Faculty of Computer Science and Mathematics

Exposé

Design and Implementation of a Comprehensive GUI and Backend System for Physical Unclonable Function (PUF) Analysis

Master's Thesis

Author: Muhammad Sohaib Akhtar
Matriculation Number: 115151
Supervisor: Dr. Nikolaos Athanasios Anagnostopoulos
Date: October 6, 2025

Contents

1 Introduction

Physical Unclonable Functions (PUFs) have emerged as one of the most promising hardware security primitives in recent years, particularly for IoT and embedded systems security. These unique device fingerprints exploit manufacturing variations that are inherent in semiconductor fabrication processes, making them practically impossible to clone or predict. However, the lack of proper comprehensive analysis and configuration tools with modern user interfaces is a challenge to be tackled.

Current PUF analysis systems often rely on basic command-line interfaces, which impose significant limitations. For example, SPAT (Statistical PUF Analysis Tool) from Sandia Labs exists [1], lacks interaction, configuration options, and storage capabilities for results.

This thesis aims to bridge this gap by developing a comprehensive GUI and backend system that not only visualizes PUF responses but also provides advanced analytical capabilities and operations, secure data management, and a user-friendly interface for both researchers and practitioners. The proposed system will go beyond basic demonstrations to include features like asynchronous processing, comprehensive testing frameworks, user data management, user sessions, and advanced visualization techniques that have not been thoroughly explored in existing literature or practice.

1.1 Literature Review

The field of PUF analysis systems has seen various developments over the years, but most focus on the hardware aspects rather than feature-complete software solutions. According to recent surveys, FPGA-based PUF implementations have gained significant traction, with over several relevant publications in the field. However, the software infrastructure for analyzing these implementations remains relatively underdeveloped and untapped for maximum potential given modern technology.

1.2 Research Gap

While hardware implementations of PUFs are well-documented, there is a clear gap in the literature regarding comprehensive software systems for PUF analysis. Most existing command-line solutions either focus on specifics (like ECC) or provide only basic functionality. The integration of modern technologies, advanced analysis capabilities, and a production-ready app for PUF analysis remains largely unexplored.

2 Approach

2.1 Research Question

How can we design and implement a comprehensive and user-friendly GUI and backend system for PUF analysis that addresses the limitations of existing tools while providing advanced PUF operations, PUF analytics and maintaining security requirements?

2.2 Hypothesis

We hypothesize that by leveraging modern architecture, and advanced analytical techniques, we can create a PUF analysis solution that:

- Significantly improves the user experience compared to existing command-line tools
- Enables complex analysis of PUF datasets and allows for parallelization of complex computations
- Provides secure multi-user access with proper authentication and user data management
- Offers extensible architecture for future extensions and additions

2.3 Methodology

The research will follow an iterative development approach based on the MVP (Minimum Viable Product) principle. The methodology consists of:

2.3.1 Requirements Analysis

Initially, we will proceed with extracting pain points from the existing solutions. This will help us define solid foundational requirements. Later, User Acceptance Testing (UAT) will help us improve further.

2.3.2 System Architecture Design

The system will be designed using clean architecture practices with clear separation between frontend, backend, and data processing components. This approach ensures maintainability and perfect separation of concerns.

2.3.3 Implementation Strategy

Development will proceed in iterative sprints, with each 3-week sprint delivering a major functional increment. The technology stack will include:

- Frontend: Modern Typescript framework in addition to modern and advanced charting libraries
- Backend: RESTful API endpoints with asynchronous processing capabilities and authentication
- Database: Lightweight solution for storing PUF responses and analysis results with offline support
- Testing: Comprehensive unit and integration testing

2.3.4 Evaluation

The system will be evaluated through:

- Performance and feature benchmarks compared with existing solutions
- User acceptance testing with target user groups
- Security assessment following strict industry-grade guidelines
- Usability tests with large PUF datasets

3 Schedule

The project timeline spans 6 months, organized into MVP iterations. Each phase builds upon the previous one (see Table 1 below).

4 Expected Outcomes

Upon completion, this thesis will deliver:

1. A production-ready application for PUF analysis
2. Comprehensive documentation and setup guides
3. Performance and feature set evaluation results compared with existing solutions
4. A strong foundation for future research directions and extensions

The system will significantly advance beyond traditional command-line toolkits by providing researchers and practitioners with a modern, feature-rich tool for PUF analysis.

Table 1: Project Schedule - MVP Approach (3-Week Sprints)

Sprint	Deliverables	Duration	Timeline
Sprint 1: Foundation & Initial Development			
MVP 1	Extended requirements analysis, existing literature review, stack finalization, basic backend setup with database schema and API endpoints	3 weeks	Weeks 1–3
Sprint 2: Core Frontend & Base Authentication			
MVP 2	Extended frontend development with routing, PUF visualization canvas and layouts, user authentication setup (static), models, updated DTOs, custom hooks	3 weeks	Weeks 4–6
Sprint 3: Visualization & UI additions			
MVP 3	Advanced visualization features (e.g., line graphs, box plots, interactive charts, etc.), real-time logging, progress tracking, electron bundle optimization	3 weeks	Weeks 7–9
Sprint 4: Asynchronous Processing & Performance			
MVP 4	Asynchronous processing with job queues, worker implementation, performance optimization, caching strategies	3 weeks	Weeks 10–12
Sprint 5: Testing & CI/CD			
MVP 5	Comprehensive unit tests (70% coverage at least), integration tests, load tests, GitLab CI pipeline setup	3 weeks	Weeks 13–15
Sprint 6: Advanced Features & Documentation			
MVP 6	Additional database entities, advanced analytics features, export functionality, advanced integrations (e.g. fim-git auth), documentation kickoff	3 weeks	Weeks 16–18
Sprint 7: UAT & Refinement			
MVP 7	User acceptance testing, feedback implementation, UI/UX refinements, performance tuning (if needed), security audit, feature requests, first thesis draft	2 weeks	Weeks 19–20
Sprint 8: Documentation & Thesis			
Final	Complete review of documentation, thesis review and finalization, evaluation results compilation, presentation preparation	4 weeks	Weeks 21–24

References

[1] Sandia National Laboratories. (2021). SPAT: Statistical PUF Analysis Tool. GitHub Repository.

[2] Cortez, M., et al. (2014). Testing PUF-Based Secure Key Storage Circuits. Design, Automation & Test in Europe Conference.

[3] Sengar, G., et al. (2018). PUF-Based Secure Test Wrapper for SoC Testing. IEEE International Test Conference.